

LAB EXPERIMENTS

Q21) Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

Aim:

To write a C program to demonstrate the **padding technique** used in ECB, CBC, and CFB modes by appending a **1 bit followed by zero bits** to make the plaintext length a multiple of the block size.

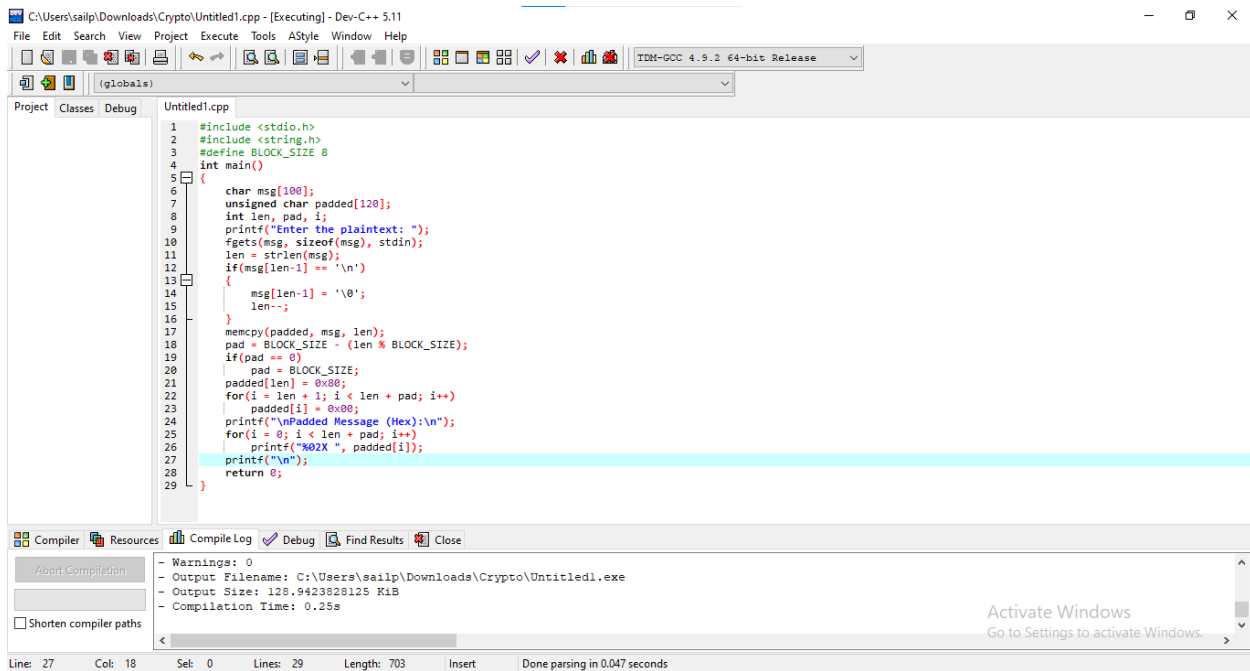
Algorithm:

1. Start the program.
2. Read the plaintext message from the user.
3. Specify the block size (e.g., 8 bytes).
4. Calculate the length of the plaintext.
5. Determine the number of padding bytes required.
6. If the plaintext length is already a multiple of the block size, add an extra block of padding.
7. Append one padding byte (0x80, representing a 1 bit followed by zeros).
8. Fill the remaining padding bytes with 0x00.
9. Display the padded plaintext in hexadecimal format.
10. Stop the program.

Result:

Thus, the C program to demonstrate **message padding** for **ECB, CBC, and CFB modes** was successfully executed. The plaintext was padded using the **1-bit followed by zero bits** method to ensure that its length became a multiple of the block size.

Code:

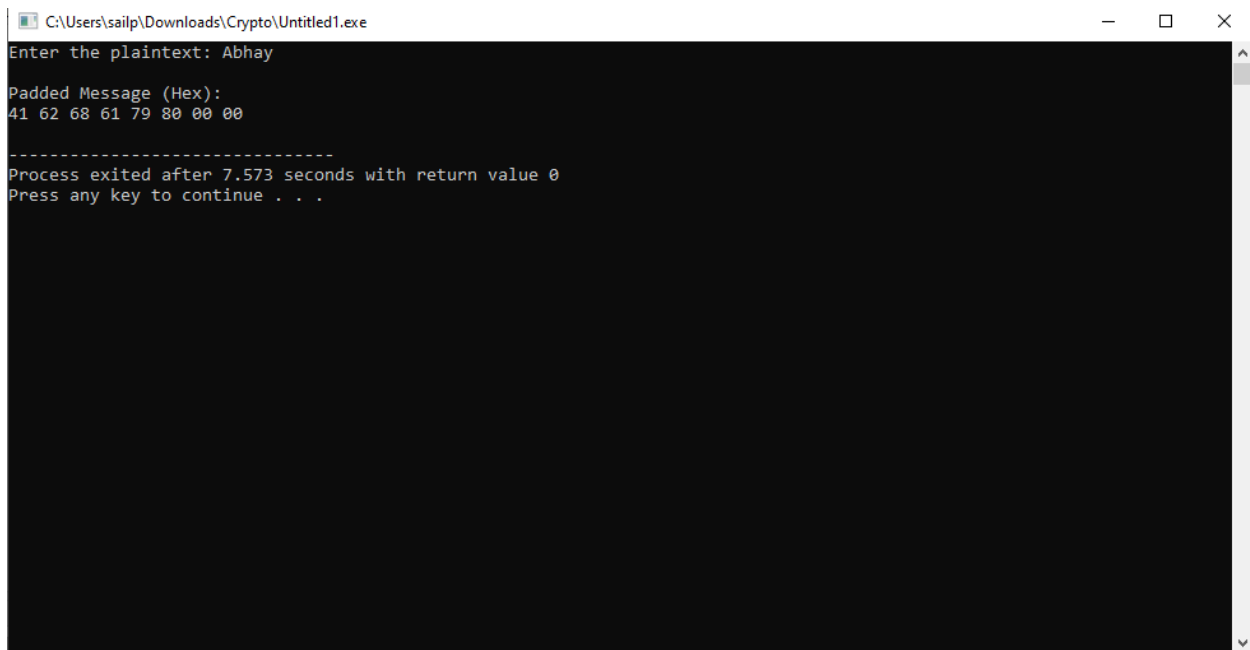


```
1 #include <stdio.h>
2 #include <string.h>
3 #define BLOCK_SIZE 8
4 int main()
5 {
6     char msg[100];
7     unsigned char padded[120];
8     int len, pad, i;
9     printf("Enter the plaintext: ");
10    fgets(msg, sizeof(msg), stdin);
11    len = strlen(msg);
12    if(msg[len-1] == '\n')
13    {
14        msg[len-1] = '\0';
15        len--;
16    }
17    memcpy(padded, msg, len);
18    pad = BLOCK_SIZE - (len % BLOCK_SIZE);
19    if(pad == 0)
20        pad = BLOCK_SIZE;
21    padded[len] = 0x80;
22    for(i = len + 1; i < len + pad; i++)
23        padded[i] = 0x00;
24    printf("\nPadded Message (Hex):\n");
25    for(i = 0; i < len + pad; i++)
26        printf("%02X ", padded[i]);
27    printf("\n");
28    return 0;
29 }
```

Compiler Warnings: 0
Output Filename: C:\Users\sailp\Downloads\Crypto\Untitled1.exe
Output Size: 128.9423828125 KiB
Compilation Time: 0.25s

Line: 27 Col: 18 Sel: 0 Lines: 29 Length: 703 Insert Done parsing in 0.047 seconds

Output:



```
C:\Users\sailp\Downloads\Crypto\Untitled1.exe
Enter the plaintext: Abhay

Padded Message (Hex):
41 62 68 61 79 80 00 00

-----
Process exited after 7.573 seconds with return value 0
Press any key to continue . . .
```